A–Z Complete Trading Platform

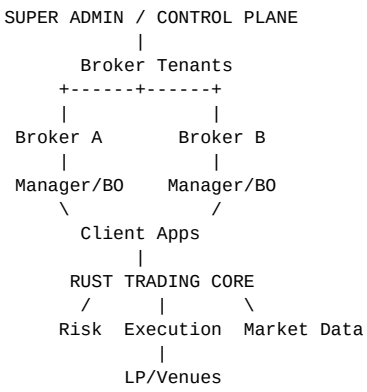
Master Architecture & Development Blueprint

Purpose: Master specification for a multi-tenant broker/trading platform. Give this document to Claude Code before implementation. Build it phase-by-phase; do not implement the entire system in one pass.

Primary goals: financial correctness, security, low-latency execution, real-time UX, multi-tenancy, auditability, scalability and maintainability.

1. Product Scope

Build: Client Mobile Terminal, Client Desktop Terminal, Client WebTrader, Broker Manager Terminal, Broker Back Office, Super Admin Terminal for registering/managing broker tenants, plus centralized Trading Core, Market Data, Risk, Execution and Liquidity Provider integrations.



2. Non-Negotiable Rules

• Rust Trading Core owns trading, risk, margin, P/L and execution authority.

• Do not put every dataset into one database.

• PostgreSQL = durable business/financial source of truth.

• Redis = cache/temporary shared state, never the financial ledger.

• ClickHouse = high-volume market data and analytics.

• NATS = event bus, not a database.

• Critical execution paths use Rust in-memory state and avoid unnecessary DB round trips.

• All financial operations are transactional, auditable and idempotent.

• Every tenant-scoped record/action must be isolated by tenant_id and server-side authorization.

• Clients never connect directly to databases or LPs.

3. Technology Stack

Rust: Trading Core, Risk, Execution, Market Data, LP adapters. **TypeScript + Node.js:** business/API services. **Next.js + React + TypeScript:** WebTrader, Manager, Back Office, Super Admin. **Tauri + React + TypeScript:** Desktop client. **Flutter + Dart:** Mobile. **PostgreSQL:** durable data. **Redis:** fast state/cache. **NATS:** events. **ClickHouse:** ticks/analytics. **Docker + Linux/Ubuntu LTS:** deployment. **Prometheus + Grafana + centralized logs:** observability. **GitHub Actions/equivalent:** CI/CD.

Electron is not part of the production desktop architecture. Use Tauri unless a documented requirement proves otherwise.

4. Client Mobile Terminal

Auth: registration/login where enabled, 2FA, biometric unlock, password recovery, device/session management. Account: balance, equity, free margin, margin level, floating P/L, account switching. Markets: search, watchlists, Bid/Ask, spread, daily change, status. Charts: candles, timeframes, indicators, drawings, crosshair, templates. Trading: market/pending orders, volume, SL/TP, modify, partial close, close, reverse where supported, one-click trading. History: orders, deals, positions, P/L,

commission, swap, deposits/withdrawals. Alerts: fills, SL/TP, margin, stop-out, price alerts. Profile/KYC/security.

5. Client Desktop Terminal — Tauri

Professional workstation: Market Watch, multi-chart layouts, multiple monitors, one-click trading, advanced order ticket, positions/orders/history, indicators, drawings, templates, alerts, hotkeys, workspace profiles and account switching. Future: algorithmic SDK, custom indicators, strategy tester/backtesting and trading API. Desktop never connects directly to PostgreSQL/LPs.

6. Client WebTrader

Next.js/React terminal with watchlists, charts, order ticket, positions, pending orders, history, account metrics, alerts and responsive layouts. Reuse API types and safe UI packages, but never client-side secrets.

7. Broker Manager Terminal

Dashboard: clients, active accounts, online users, positions, pending orders, exposure, floating P/L, volume, revenue, margin calls, stop-outs, LP health and execution latency.

Accounts: create/activate/suspend/freeze, reset credentials, change leverage/group, assign IB, view history/positions, restrictions. Dealing: close/partial close, SL/TP modification, pending cancellation and emergency controls; every action goes through Trading Core.

Risk: account/group/symbol/broker limits, max lot/exposure/positions, daily loss/drawdown, leverage, margin call, stop-out, buy-only/sell-only, sessions. Symbols: contract size, min/max lot, lot step, digits, tick size/value, margin, commission, swap, spread/markup, sessions, order types. Groups: Standard, VIP, ECN, Professional, Demo, Prop and custom. Execution: LP health, latency, rejects, fills, slippage, mapping and routing configuration. Exposure by symbol/account/group/IB.

8. Broker Back Office

CRM, client profiles, accounts, KYC/AML, deposits, withdrawals, finance, ledger views, IB/affiliate, reports, support, notifications, admin users and audit. Finance states: pending/approved/rejected/processing/completed. Maker/checker approval can be required for sensitive actions. Reports: volume, lots, P/L, commission, swap, revenue, deposits, withdrawals, exposure, active clients and IB performance; CSV/XLSX/PDF exports.

9. Super Admin Terminal — Platform Owner

Super Admin is the platform control plane, not a broker admin. Features: broker onboarding, tenant creation, company profile, primary admin, domain/branding, timezone/currency, legal settings, plan/subscription, feature flags and environment status.

Tenant lifecycle: trial, active, read-only, suspended, disabled, archived. Manage tenant admins, quotas, enabled modules, instruments, API limits, support access and service health. Global controls: supported symbol catalog, LP connectors, payment/notification providers, feature flags and version management.

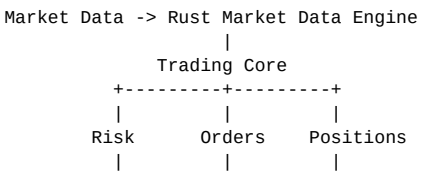
Platform monitoring: tenant health, Trading Core, API, WebSockets, LP connectivity, message rates, DB/storage health and incidents. Optional commercial module: plans, billing status, invoices, usage and entitlements. Billing stays separate from the trading ledger. Cross-tenant access is explicit, least-privilege and fully audited.

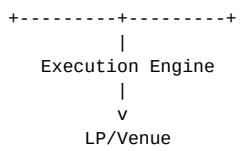
10. Multi-Tenant Model

```
Tenant A: tenant_id=A
  users/accounts/orders/deals/ledger/config/reports
Tenant B: tenant_id=B
  users/accounts/orders/deals/ledger/config/reports
```

Every tenant-scoped record must carry tenant_id or be reached only through a strictly tenant-scoped relationship. Enforce tenant authorization server-side; use PostgreSQL Row Level Security where practical as defense-in-depth.

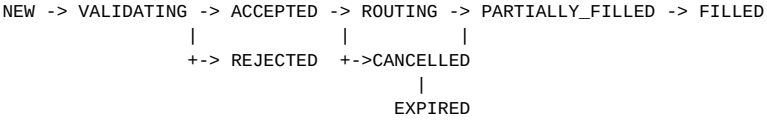
11. Central Rust Trading Core





Trading Core owns order validation/state transitions, positions, margin, exposure, risk, P/L, execution state and trading permissions.

12. Order & Position Lifecycle



State transitions are explicit, validated and idempotent. UI code cannot arbitrarily mutate trading state.

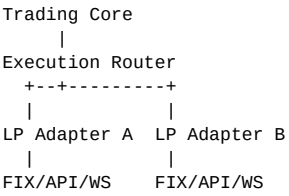
13. Risk & Margin

Support account/group/symbol/broker risk rules, leverage, margin, free margin, margin level, margin call, stop-out, max exposure, max volume, max positions, daily loss/drawdown, trading permissions and emergency halt. Risk is authoritative in Rust; Manager configures policy.

14. Market Data

LP/venue feeds enter Rust adapters, are normalized, held as hot market state in memory, published through NATS, streamed through WebSockets and persisted historically to ClickHouse. Do not synchronously write every tick to PostgreSQL.

15. LP / Execution Architecture



Use a provider-agnostic adapter interface. Support future FIX/REST/WebSocket connectors. Securely store credentials. Track symbol mapping, connection health, latency, reject/fill/slippage metrics and routing policies. Manager configures; Execution Engine executes.

16. Data Architecture

```
POSTGRESQL — durable truth
 users, tenants, accounts, orders/deals, ledger,
 deposits/withdrawals, KYC metadata, groups/symbol config,
 IB, permissions, audit

REDIS — fast/temporary
 sessions, cache, rate limits, locks, temporary/shared state

CLICKHOUSE — high-volume analytics
 ticks, candles, execution analytics, time-series reports

RUST MEMORY — ultra-low-latency state
 current prices, active orders/positions, exposure, margin/risk

NATS — event bus
 domain events/messages; NOT permanent storage
```

17. Financial Ledger

Never directly overwrite balances. Every balance-changing operation creates an immutable ledger transaction. Corrections are compensating transactions. Use decimal/fixed-point money types, ACID transactions, unique idempotency keys and strict authorization.

```
Deposit -> Ledger -> Account Financial State
Trade -> Fill -> P/L + Commission + Swap -> Ledger
Withdrawal -> Approval -> Ledger -> Payment State
```

18. API & Real-Time

REST/HTTP for CRUD, configuration and reporting. WebSockets for prices, orders, positions, account metrics, exposure and notifications. NATS for internal domain events. API Gateway enforces auth, tenant scope, RBAC, validation, rate limits, idempotency and correlation IDs.

19. Authentication / RBAC

Short-lived access tokens, refresh rotation, 2FA, secure cookies where appropriate, session/device management and brute-force protection. Broker roles: SUPER_ADMIN (tenant), ADMIN, MANAGER, DEALER, RISK_MANAGER, FINANCE, SUPPORT, IB_MANAGER, KYC_OFFICER, AUDITOR. Platform Super Admin roles are separate and platform-scoped.

20. Security

TLS everywhere, secret management, least privilege, private DB networks, firewalling, secure headers, input validation, dependency/container scanning, audit logs, IP/device controls where required, secret rotation, encrypted backups and disaster recovery. Never expose DB/LP credentials to clients.

21. Server / VPS Topology

```
SERVER A — Trading
  Rust Core | Risk | Execution | Market Data | NATS

SERVER B — Data/Backend
  PostgreSQL | Redis | Node.js business/API services

SERVER C — Application (scale-out)
  WebSocket Gateway | Manager | Back Office | WebTrader edge | reports

SERVER D — Optional
  ClickHouse / analytics

Super Admin = protected control-plane application;
no direct browser-to-database access.
```

Place trading infrastructure near the relevant LP/venue. Keep database/backend in low-latency private networking with the core. Scale API/app tiers horizontally as traffic grows.

22. Observability

Monitor order/risk/execution/market-data latency, WebSocket/API latency, errors, DB latency, CPU/RAM/disk/IOPS, NATS rates, Redis latency, LP rejects/fills/slippage and tenant health. Use Prometheus/Grafana, structured centralized logs, correlation IDs and alerts.

23. Backup / Disaster Recovery

PostgreSQL: automated backups, point-in-time recovery where appropriate, off-server backups and restore tests. Define RPO/RTO. Design for DB outage, service restart, duplicate events, LP timeout and WebSocket disconnect. Financial operations must be recoverable and idempotent.

24. Testing

Unit: P/L, margin, commission, swap, risk, ledger, state machines, permissions. Integration: API -> Core -> Risk -> Execution -> Ledger. E2E: login -> order -> fill -> position -> close -> ledger. Load: ticks, concurrent users, orders, WebSockets. Failure: LP disconnect, DB outage, duplicate event, restart.

25. Monorepo / Folder Architecture

```
/apps
  /client-webtrader
  /client-desktop
  /client-mobile
  /broker-manager
  /broker-backoffice
  /super-admin

/services
  /api-gateway
  /auth
```

```
  /finance
  /crm
  /kyc
  /reporting
  /notifications
  /websocket-gateway

/core-rust
  /trading-core
  /risk-engine
  /execution-engine
  /market-data
  /lp-adapters

/packages
  /api-types
  /event-types
  /permissions
  /ui
  /validation
  /config

/infra
  /docker
  /deployment
  /monitoring

/docs
  architecture.md
  database.md
  api.md
  events.md
  security.md
  rbac.md
  execution.md
  risk.md
  tenancy.md
```

Adapt to the existing repository; do not duplicate working modules just to match this example.

26. A–Z Development Roadmap

Phase 0 — Architecture Lock

Inspect repository; finalize boundaries, data ownership, tenant model, API/event contracts, ERD, security and deployment.

Phase 1 — Foundation

Monorepo, Rust workspace, TypeScript services, Flutter/Tauri shells, PostgreSQL migrations, Redis, NATS, auth/RBAC, Docker and CI.

Phase 2 — Trading Core

Accounts, symbols, orders, deals, positions, state machines, P/L, commission, swap, margin and risk.

Phase 3 — Market Data

LP adapters, normalization, hot state, WebSocket streaming and ClickHouse persistence.

Phase 4 — Execution

Execution abstraction, LP connectors, routing, fills, slippage/reject handling and execution metrics.

Phase 5 — Client Products

Desktop, WebTrader and Mobile core trading workflows.

Phase 6 — Broker Manager

Dealing desk, risk, exposure, groups, symbols, execution monitoring and account operations.

Phase 7 — Back Office

CRM, KYC, finance, ledger views, deposits/withdrawals, IB, reports and audit.

Phase 8 — Super Admin

Broker onboarding, tenant management, plans/features, platform monitoring, support access and global controls.

Phase 9 — Hardening

Load, security, recovery, observability, backups, performance tuning and production deployment.

Phase 10 — Scale

Horizontal API scaling, read replicas where justified, ClickHouse scaling, additional LPs/regions, copy trading, PAMM/MAM, algorithmic APIs and strategy tester.

27. Claude Code Rules

• Do not implement the whole platform in one prompt.

• Do not change core technology choices without presenting trade-offs and receiving approval.

• Do not use Electron in production desktop; use Tauri.

• Do not put all data in PostgreSQL.

• Do not put trading logic in React/Next.js/Flutter.

• Do not allow UI clients to access databases or LPs directly.

• Do not bypass Trading Core for Manager manual actions.

• Do not mutate financial balances without ledger transactions.

• Do not store secrets in source control.

• Do not rewrite working code unnecessarily.

• Before each major phase: inspect, document interfaces, implement, test, security-review and benchmark where relevant.

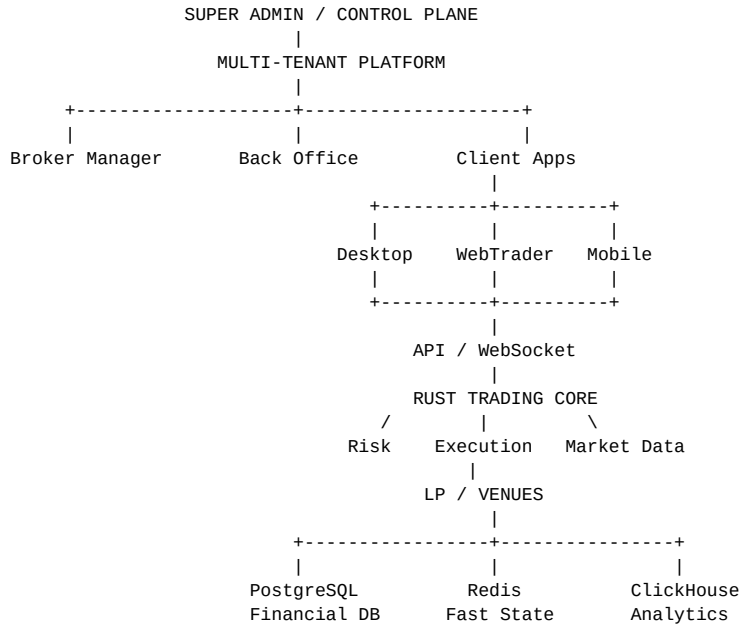
28. First Claude Code Task

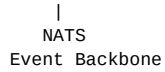
DO NOT START CODING THE ENTIRE PLATFORM. First inspect the repository and produce an Architecture Review against this document: existing modules, missing modules, contradictions, migration risks, final ERD, service boundaries, tenant isolation, API outline, event outline, Rust module boundaries, LP adapter interface, infrastructure topology, security model and phased plan. Wait for approval before major implementation.

29. Definition of Done

A feature is complete only when implementation, automated tests, authorization/security, error handling, observability, audit requirements, migrations, documentation and rollback/recovery considerations are addressed. No critical security, financial-consistency or data-loss issue.

30. Final Target





Final principle: optimize for correctness first, then latency, then scale. Never sacrifice financial integrity for superficial UI speed.